

OWL PRINT — Manual de Instalação (VS Code)

Guia passo a passo para rodar o sistema **OWL PRINT — Cardápios Personalizados** na sua máquina usando o **Visual Studio Code**, e publicá-lo na internet (Vercel).

Você recebeu dois itens: 1. **owl-print/** — o sistema completo (site público + ERP) em Next.js. Roda de verdade, com banco de dados e login. 2. **owl-print-demo.html** — protótipo offline de uma página só, para visualizar o front-end e a navegação. Não precisa instalar nada: é só dar duplo-clique.

Sumário

- 1. Pré-requisitos
- 2. Instalar o Node.js
- 3. Instalar o VS Code e extensões
- 4. Abrir o projeto
- 5. Instalar as dependências
- 6. Configurar as variáveis de ambiente (.env.local)
- 7. Rodar em desenvolvimento
- 8. Build de produção
- 9. Publicar na internet (Vercel)
- 10. Abrir o protótipo offline
- 11. Estrutura de pastas
- 12. Solução de problemas
- 13. Checklist final

1. Pré-requisitos

Item	Para quê	Custo
Node.js 18+ (LTS)	rodar o projeto	grátis
VS Code	editar e rodar	grátis
Conta MongoDB Atlas	banco de dados	grátis (plano M0)
Conta Auth0	login do painel admin	grátis
Conta Cloudinary	imagens dos produtos/portfólio	grátis
Conta Resend (opcional)	enviar orçamento por e-mail	grátis
Conta Vercel (opcional)	publicar online	grátis

2. Instalar o Node.js

- 1. Acesse **https://nodejs.org** e baixe a versão **LTS**.
- 2. Instale (avançando com "Next/Próximo").
- 3. Para conferir, abra o **Prompt de Comando** (Windows) ou **Terminal** (Mac) e digite:

```
node -v
npm -v
```

Devem aparecer números de versão (ex.: **v20.x.x**). Se aparecer, está ok.

3. Instalar o VS Code e extensões

- 1. Baixe em **https://code.visualstudio.com** e instale.
- 2. Abra o VS Code, clique no ícone de **Extensões** (quadrinhos na barra lateral) e instale:
- 3. **ESLint** (dbaeumer.vscode-eslint)
- 4. **Tailwind CSS IntelliSense** (bradlc.vscode-tailwindcss)
- 5. **Prettier - Code formatter** (esbenp.prettier-vscode)

Essas extensões ajudam com autocompletar e formatação, mas não são obrigatórias para rodar.

4. Abrir o projeto

1. **Descompacte** o arquivo recebido em uma pasta de fácil acesso (ex.: `Documentos/owl-print`).
2. No VS Code: menu **Arquivo → Abrir Pasta...** e selecione a pasta `owl-print` (a que contém o arquivo `package.json`).
3. Abra o terminal integrado: menu **Terminal → Novo Terminal** (ou `Ctrl + Crase`).

5. Instalar as dependências

No terminal integrado (já dentro da pasta do projeto), rode:

```
npm install
```

Aguarde (pode levar 1–3 minutos). Isso baixa o Next.js, React, Mongoose e tudo o mais.

6. Configurar as variáveis de ambiente (.env.local)

O projeto vem com um modelo chamado `.env.local.example` . Faça uma cópia dele chamada `.env.local` e preencha.

No terminal:

```
# Mac/Linux
cp .env.local.example .env.local
# Windows (PowerShell)
copy .env.local.example .env.local
```

Agora abra o `.env.local` no VS Code e preencha cada serviço:

6.1 MongoDB Atlas (banco de dados)

1. Crie conta em <https://www.mongodb.com/atlas> e um **cluster gratuito (M0)**.
2. Em **Database Access**, crie um usuário e senha.
3. Em **Network Access**, adicione o IP `0.0.0.0/0` (libera acesso de qualquer lugar — ok para começar).
4. Em **Database → Connect → Drivers**, copie a *connection string* e cole em:

```
MONGODB_URI=mongodb+srv://USUARIO:SENHA@cluster.mongodb.net/owlprint?retryWrites=true&w=majority
```

Troque `USUARIO` e `SENHA` pelos que você criou.

6.2 Auth0 (login do painel admin)

1. Crie conta em <https://auth0.com> → **Applications → Create Application → Regular Web Application**.
2. Em **Settings**, configure:
3. **Allowed Callback URLs:** `http://localhost:3000/api/auth/callback`
4. **Allowed Logout URLs:** `http://localhost:3000`
5. Copie **Domain**, **Client ID** e **Client Secret** para:

```
AUTH0_ISSUER_BASE_URL=https://SEU-DOMINIO.us.auth0.com
AUTH0_CLIENT_ID=...
AUTH0_CLIENT_SECRET=...
AUTH0_BASE_URL=http://localhost:3000
```

1. Gere um segredo aleatório para `AUTH0_SECRET` :

```
# Mac/Linux
openssl rand -hex 32
```

No Windows, use qualquer texto longo e aleatório (32+ caracteres) ou gere em <https://generate-secret.vercel.app/32>.

```
AUTH0_SECRET=cole-o-valor-gerado-aqui
```

6.3 Cloudinary (imagens)

1. Crie conta em <https://cloudinary.com>. No **Dashboard** aparecem **Cloud name**, **API Key** e **API Secret**.
2. Em **Settings → Upload → Upload presets**, crie um preset com **Signing Mode = Unsigned** e nome `owlprint_unsigned` .
3. Preencha:

```
CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=...
NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET=owlprint_unsigned
```

6.4 E-mail (opcional — enviar orçamento por e-mail)

1. Crie conta em **https://resend.com**, gere uma **API Key**.

```
RESEND_API_KEY=...
EMAIL_FROM=OWL PRINT <orcamentos@seudominio.com.br>
```

Sem isso, o envio por e-mail apenas registra no console (o sistema continua funcionando).

6.5 Analytics (opcional)

```
NEXT_PUBLIC_GA_ID=G-XXXXXXXXXX
NEXT_PUBLIC_META_PIXEL_ID=...
NEXT_PUBLIC_WHATSAPP_NUMBER=5511999999999
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Importante: sempre que você editar o `.env.local`, pare o servidor (`Ctrl + C` no terminal) e rode `npm run dev` de novo.

7. Rodar em desenvolvimento

No terminal:

```
npm run dev
```

Abra o navegador em **http://localhost:3000**.

- **Site público:** `http://localhost:3000`
- **Painel administrativo:** `http://localhost:3000/admin` (vai pedir login do Auth0)

Para parar o servidor: clique no terminal e pressione `Ctrl + C`.

7.1 (Opcional) Popular o banco com dados de exemplo

O projeto inclui um script que preenche o banco com produtos, clientes, orçamentos, pedidos, financeiro e portfólio de exemplo — útil para já ver o painel cheio. Com o `MONGODB_URI` configurado, rode:

```
npm run seed
```

Atenção: o seed **apaga e recria** as coleções de exemplo. Use apenas para popular um banco novo/de testes.

8. Build de produção (testar a versão final)

```
npm run build
npm start
```

O `build` confere se está tudo certo (tipos, etc.). Se aparecer algum erro, ele será mostrado no terminal indicando o arquivo.

9. Publicar na internet (Vercel)

A forma mais simples de deixar o sistema no ar:

1. Crie um repositório no **GitHub** e envie o projeto (pode usar o painel **Source Control** do VS Code).
2. Crie conta em **https://vercel.com** e clique em **Add New → Project → Import** (escolha o repositório).
3. Em **Environment Variables**, adicione **as mesmas variáveis** do seu `.env.local` (troque as URLs `localhost:3000` pela URL do seu site, ex.: `https://owlprint.vercel.app`).

- 4. No Auth0, adicione também essa URL nas **Callback/Logout URLs**.
- 5. Clique em **Deploy**. Em ~1 minuto o site estará no ar com endereço público.

10. Abrir o protótipo offline

Quer só ver o visual e a navegação, sem instalar nada?

- Dê **duplo-clique** em `owl-print-demo.html` — abre no navegador, funciona **offline** e no celular.
- O painel admin do protótipo abre com login de demonstração:
- **Usuário:** `admin`
- **Senha:** `owlprint`

O protótipo **salva o que você cadastrar no próprio navegador** (não perde ao recarregar). Para voltar aos dados de exemplo, use "**🔄 Reiniciar dados**" na barra lateral do admin. É uma demonstração visual/funcional — o sistema "de produção" é o da pasta `owl-print/`, com banco de dados real.

11. Estrutura de pastas (resumo)

```
owl-print/
├─ src/
│  ├─ app/           # páginas (site público + /admin) e rotas de API
│  │  ├─ admin/      # Dashboard, Orçamentos, CRM, Produção, Financeiro, Produtos, Portfólio
│  │  ├─ produtos/   # catálogo público
│  │  ├─ portfolio/  # portfólio público
│  │  └─ api/        # rotas de API (auth, leads, export)
│  ├─ actions/       # regras de negócio (server actions)
│  ├─ components/    # componentes de interface
│  ├─ models/        # modelos do banco (Mongoose)
│  ├─ lib/           # conexão com o banco, utilidades
│  └─ types/         # tipagens e rótulos em PT-BR
├─ .env.local.example # modelo de variáveis
├─ package.json       # dependências e comandos
└─ tailwind.config.ts  # identidade visual (cores, fontes)
```

12. Solução de problemas

"command not found: npm" / "node não é reconhecido" → O Node.js não foi instalado ou o terminal não foi reaberto. Feche e abra o VS Code novamente após instalar o Node.

A porta 3000 já está em uso → Rode em outra porta: `npm run dev -- -p 3001` e acesse `http://localhost:3001`.

Erro de conexão com o MongoDB → Confira a senha na `MONGODB_URI` e se você liberou o IP `0.0.0.0/0` em *Network Access* no Atlas.

Login não funciona / erro de callback → Verifique se a **Callback URL** no Auth0 é exatamente `http://localhost:3000/api/auth/callback` e se `AUTH0_SECRET` está preenchido.

Upload de imagem falha → Confirme que o *upload preset* do Cloudinary está como **Unsigned** e com o mesmo nome de `NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET`.

As telas do admin aparecem vazias → Normal no começo: o banco está sem dados. Cadastre um produto, um lead e um orçamento pelo próprio painel.

13. Checklist final

- [] Node.js instalado (`node -v` funciona)
- [] Projeto aberto no VS Code
- [] `npm install` concluído
- [] `.env.local` criado e preenchido (MongoDB, Auth0, Cloudinary)
- [] `npm run dev` rodando
- [] Site abre em `http://localhost:3000`
- [] Login do admin funciona em `/admin`

Pronto! Qualquer erro que aparecer no terminal, copie a mensagem — ela sempre indica o arquivo e a linha do problema.